

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-2 Créditos: 3	
---	---	---

Taller 1: Búsqueda con y sin información

Contexto general

La estación espacial Aurora cumple funciones de investigación, comunicación y monitoreo orbital. Durante una tormenta de partículas, varios sistemas de energía y control presentan fallas simultáneas. Algunas secciones permanecen operativas, mientras que otras tienen corredores deteriorados, zonas de baja gravedad, sectores con radiación elevada y compuertas bloqueadas.

Debido a estas condiciones, el personal no puede desplazarse de manera segura entre los módulos. Para atender la emergencia se activa un robot autónomo de mantenimiento, cuya misión consiste en navegar por la estación, diagnosticar fallas, transportar componentes de reparación y restaurar sistemas afectados.

El robot conoce la estructura general de la estación, pero debe calcular las rutas que seguirá. A lo largo del taller, la misión aumenta progresivamente en complejidad: primero debe alcanzar una terminal de diagnóstico, luego comparar rutas por cantidad de pasos y por costo total, después transportar un módulo obligatorio y finalmente reparar varios sistemas usando búsqueda informada.

La estación se modela como una cuadrícula compuesta por corredores, muros, terminales, módulos de energía, kits de reparación, centros de control y sistemas averiados. Esta representación permite formular cada misión como un problema de búsqueda con estados, acciones, sucesores, metas y costos.

Objetivos de aprendizaje

Al completar este taller, el estudiante será capaz de:

1. Implementar y comparar DFS, BFS, UCS y A* en problemas de navegación sobre cuadrículas.
2. Diferenciar entre encontrar cualquier solución, minimizar el número de movimientos y minimizar el costo total de una ruta.
3. Usar correctamente la interfaz de un problema de búsqueda: estado inicial, prueba de meta, sucesores y costo de acciones.
4. Modelar un problema con estado compuesto, submeta obligatoria y costo dependiente del estado.
5. Diseñar, implementar y comparar heurísticas para A*, conservando la optimalidad del algoritmo y analizando su efecto sobre la cantidad de nodos expandidos.
6. Analizar los algoritmos en términos de completitud, optimalidad, complejidad, costo de la solución, nodos expandidos y tiempo de ejecución.

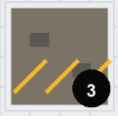
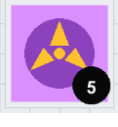
Escenario del problema

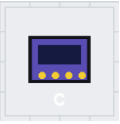
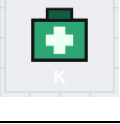
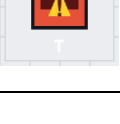
Para todas las misiones del taller, el robot se mueve dentro de una cuadrícula. Cada celda representa un corredor, un muro o una ubicación especial de la estación. Las reglas generales son las siguientes:

- Puede moverse únicamente hacia arriba, abajo, izquierda y derecha.
- No puede atravesar muros, compuertas bloqueadas ni salir de los límites del mapa.
- Cada movimiento tiene un costo determinado por el corredor al que entra, excepto cuando la misión indique costo uniforme.
- Puede recoger componentes y reparar sistemas al alcanzar las ubicaciones correspondientes, siempre que se cumplan las condiciones de la misión.

Los puntos 1, 2 y 3 utilizan el mismo grupo de mapas de diagnóstico, lo que permite comparar directamente el comportamiento de DFS, BFS y UCS. El punto 4 utiliza mapas de módulo y el punto 5 utiliza mapas de sistemas con dificultad creciente.

Las ubicaciones especiales D, C, M, K, T y R son transitables. Entrar a D, C, M, K o T tiene costo 1, salvo que una regla particular de la misión indique lo contrario. La ubicación R representa la posición inicial; si el robot vuelve a entrar en esta celda durante la misión, el movimiento tendrá costo 1. Algunas misiones pueden modificar el costo de movimiento según el estado del robot.

Símbolo	Elemento o tipo de corredor	Costo por paso	Descripción	Imagen
.	Corredor normal	1	Sección estable y completamente operativa. Permite el desplazamiento del robot con un consumo mínimo de energía.	
~	Zona de baja gravedad	2	Sector donde los sistemas de gravedad artificial funcionan de manera irregular. El robot debe moverse con mayor precaución y consume más energía.	
^	Corredor deteriorado	3	Sección afectada por la tormenta, con daños estructurales y obstáculos menores que dificultan el desplazamiento.	
*	Zona de radiación elevada	5	Área transitable, pero peligrosa para los componentes electrónicos del robot. Debe evitarse cuando exista una ruta de menor costo.	
%	Muro o compuerta bloqueada	∞	Estructura cerrada o dañada que el robot no puede atravesar ni ocupar.	
R	Posición inicial del robot	1	Punto desde el cual comienza la misión.	
D	Terminal de diagnóstico	1	Terminal utilizada para consultar el estado de los sistemas de la estación y realizar actualizaciones de emergencia. Es la ubicación objetivo en los puntos 1, 2 y 3.	

C	Centro de control	1	Módulo central desde el cual se coordinan las operaciones de la estación. En el punto 4 recibe el módulo de energía y, en el punto 5, corresponde a la ubicación final a la que debe llegar el robot después de completar todas las reparaciones.	
M	Módulo de energía	1	Componente que el robot debe recoger antes de dirigirse al centro de control en la misión con submeta obligatoria.	
K	Kit de reparación	1	Herramienta reutilizable que debe recogerse antes de reparar los sistemas averiados.	
T	Sistema averiado	1	Sistema que debe ser visitado y reparado. La reparación solo ocurre si el robot ya recogió el kit.	

Estructura del proyecto

El proyecto entrega una infraestructura funcional de simulación, carga de mapas, ejecución de agentes y visualización. Usted no debe reconstruir esta infraestructura. Su trabajo consiste en completar únicamente las funciones y métodos indicados en los archivos señalados a continuación.

Archivos que debe modificar

Los siguientes archivos concentran la implementación solicitada en el taller. Mantenga los nombres de funciones, clases, parámetros y formatos de retorno definidos por el proyecto.

Archivo	Propósito	Funciones o métodos por completar	Puntos
<code>algorithms/search.py</code>	Contiene los algoritmos de búsqueda que reciben una instancia de <code>SearchProblem</code> y retornan una lista de acciones.	<code>depthFirstSearch</code> , <code>breadthFirstSearch</code> , <code>uniformCostSearch</code> y <code>aStarSearch</code> .	1, 2, 3 y 5
<code>algorithms/problems.py</code>	Define las clases que modelan los problemas de búsqueda de la estación.	En <code>ModuleRepairProblem</code> : <code>isGoalState</code> , <code>_getStepCost</code> y <code>getSuccessors</code> .	4
<code>algorithms/heuristics.py</code>	Contiene las heurísticas usadas por A* en el problema de reparación de sistemas.	<code>manhattanHeuristic</code> , <code>euclideanHeuristic</code> y <code>systemRepairHeuristic</code> .	5

Otros módulos del proyecto

Los módulos siguientes hacen parte de la infraestructura. Puede consultarlos para comprender cómo se ejecuta el taller, pero no deben modificarse para resolver los puntos solicitados.

Módulo o carpeta	Uso dentro del proyecto
algorithms/agents.py	Define SearchAgent, que construye el problema indicado y llama la función de búsqueda seleccionada.
algorithms/utils.py	Contiene estructuras de datos útiles como Stack, Queue y PriorityQueue, además de utilidades del proyecto.
world/game.py	Define clases base del entorno, acciones, direcciones y utilidades como Actions.directionToVector.
world/station_layout.py	Carga los mapas y define símbolos, muros, terreno y costos de movimiento.
world/station_state.py	Mantiene el estado de la misión, la posición del robot, los símbolos y el costo acumulado mostrado en la interfaz.
world/station_rules.py	Aplica las reglas de movimiento y determina cuándo una misión se completa.
world/station_mission.py	Ejecuta una misión completa y coordina agente, estado, reglas y visualización.
view/graphics_display.py	Implementa la ventana gráfica y la animación de la solución.
view/text_display.py	Implementa la visualización en texto y el modo sin animación.
view/graphics_utils.py	Contiene primitivas auxiliares para dibujar la interfaz gráfica.
main.py	Procesa los argumentos de línea de comandos y lanza una misión concreta.
menu.py	Ofrece la interfaz de terminal recomendada para seleccionar punto, mapa, algoritmo, heurística y configuración.
layouts/diagnostic	Mapas usados en los puntos 1, 2 y 3.
layouts/module	Mapas usados en el punto 4.
layouts/systems	Mapas usados en el punto 5.

Ejecución

Antes de ejecutar el taller, instale las dependencias desde la carpeta del proyecto:

```
python -m pip install -r requirements.txt
```

La forma recomendada de ejecución es usar la interfaz de terminal:

```
python menu.py
```

Desde esta interfaz puede seleccionar el punto del taller, el mapa, el modo de animación, la velocidad, el zoom y la heurística cuando aplique. Si la interfaz de terminal no funciona en algún equipo, también puede ejecutar cada caso directamente con main.py.

La sintaxis general de ejecución por comandos es:

```
python main.py -p PROBLEM -f FUNCTION -l LAYOUT_FILE [OPCIONES]
```

Opciones de línea de comandos

Opción	Descripción	Default	Ejemplo
--help	Muestra la ayuda de ejecución y termina el programa.	No aplica	python main.py --help

-p, --problem	Selecciona la clase de problema. Valores válidos: DiagnosticProblem, ModuleRepairProblem o SystemRepairProblem.	Sin valor predeterminado, obligatorio.	-p DiagnosticProblem
-f, --function	Selecciona la función de búsqueda definida en algorithms/search.py.	Sin valor predeterminado, obligatorio.	-f breadthFirstSearch
-h, --heuristic	Selecciona la heurística para funciones que aceptan parámetro heuristic, especialmente A*.	nullHeuristic	-h systemRepairHeuristic
-l, --layout	Selecciona el archivo de layout, sin escribir la extensión .lay.	Sin valor predeterminado, obligatorio.	-l radiationShortcut
-t, --textGraphics	Ejecuta la misión con visualización por texto.	False	-t
-q, --quietTextGraphics	Ejecuta la misión con salida mínima y sin animación gráfica.	False	-q
-z, --zoom	Ajusta el tamaño de la ventana gráfica.	1.0	-z 1.25
-r, --recordActions	Guarda el historial de acciones de la misión en un archivo.	False	-r
-x, --frameTime	Define el tiempo entre cuadros. Un valor menor que 0 activa el modo paso a paso con flechas izquierda y derecha.	0.25	-x -1
-c, --catchExceptions	Activa manejo de excepciones durante la ejecución de las misiones.	False	-c

Ejemplos de comandos completos:

```
python main.py -p DiagnosticProblem -f tinyDiagnosticSearch -l tinyDiagnostic
python main.py -p DiagnosticProblem -f breadthFirstSearch -l radiationShortcut
python main.py -p ModuleRepairProblem -f uniformCostSearch -l moduleVault
python main.py -p SystemRepairProblem -f aStarSearch -h systemRepairHeuristic -l finalMaintenanceComplex
```

Ejercicios

Punto 1: Diagnóstico inicial mediante DFS (15 %)

Después de la tormenta, una terminal de diagnóstico D deja de enviar información al centro de control. El robot debe desplazarse desde su posición inicial hasta la terminal para determinar el estado de los sistemas conectados.

En esta primera misión solo se necesita encontrar cualquier camino válido hasta la terminal. Los mapas incluyen bifurcaciones, callejones y ciclos, por lo que debe controlar correctamente los estados visitados y evitar exploraciones indefinidas.

- a) Implemente `depthFirstSearch` en `algorithms/search.py`.
- b) Para esta misión, utilice la clase `DiagnosticProblem` y pruebe su implementación sobre los mapas disponibles en `layouts/ diagnostic`.

Punto 2: Ruta más corta mediante BFS (15 %)

El diagnóstico inicial confirma que la terminal D debe ser alcanzada nuevamente para realizar una actualización de emergencia. Esta vez, la ventana de comunicación con la estación es limitada, por lo que el robot debe utilizar la ruta con el menor número de movimientos.

- a) Implemente `breadthFirstSearch` en `algorithms/search.py`.
- b) Para esta misión, utilice la clase `DiagnosticProblem` y pruebe su implementación sobre los mapas disponibles en `layouts/ diagnostic`.

Punto 3: Navegación de menor costo mediante UCS (20 %)

Luego de restablecer la comunicación, los sensores identifican que varios corredores fueron afectados de forma diferente por la tormenta. Aunque todos los corredores transitables permiten llegar a la terminal D, no todos requieren la misma cantidad de energía.

El robot deberá resolver nuevamente la misión de desplazamiento entre la posición inicial y la terminal, pero ahora el objetivo será minimizar el costo total del recorrido y no la cantidad de movimientos.

- a) Implemente `uniformCostSearch` en `algorithms/search.py`.
- b) Para esta misión, utilice la clase `DiagnosticProblem` y pruebe su implementación sobre los mapas disponibles en `layouts/ diagnostic`.

Punto 4: UCS con módulo obligatorio y costo dependiente del estado (20 %)

El diagnóstico revela que el centro de control necesita un nuevo módulo de energía. El robot debe dirigirse primero al almacén de suministros, recoger el módulo M y después transportarlo hasta el centro de control C. Llegar directamente al centro de control sin haber recogido el módulo no completa la misión.

La prueba de meta será válida únicamente cuando el robot se encuentre en C y previamente haya recogido el módulo de energía M. Antes de recoger M, cada movimiento tendrá el costo normal del corredor. El movimiento utilizado para entrar en M conservará su costo normal y, a partir del movimiento siguiente, cada transición tendrá el doble del costo correspondiente.

En este punto no se implementa un nuevo algoritmo de búsqueda. Debe reutilizar el UCS del punto 3 y completar una clase de problema parcialmente implementada. El formato del estado se entrega fijo como `(position, hasModule)`, para mantener compatibilidad con el resto del proyecto.

Los métodos `getStartState` y `getCostOfActions` se entregan implementados y no deben modificarse.

- a) Complete `isGoalState` en `ModuleRepairProblem`.
- b) Complete `_getStepCost` en `ModuleRepairProblem`.
- c) Complete `getSuccessors` en `ModuleRepairProblem` respetando el formato `(nextState, direction, stepCost)`.
- d) Para esta misión, utilice la clase `ModuleRepairProblem` y resuelva los mapas disponibles en `layouts/module` mediante `uniformCostSearch`, implementado en el punto 3.

Punto 5: A* y diseño de heurísticas (20 %)

Después de restablecer el centro de control, se detectan varios sistemas secundarios averiados en distintas secciones de la estación. El robot inicia la misión en la posición R y deberá recoger un kit reutilizable de reparación K, reparar todos los sistemas averiados T y finalmente llegar al centro de control C.

El kit es obligatorio y puede utilizarse para reparar todos los sistemas. Si el robot visita un sistema averiado antes de recogerlo, la reparación no se realiza y el sistema continúa pendiente. El estado representa la posición actual, si el kit ya fue recogido y cuáles sistemas siguen pendientes. Para concentrar el ejercicio en A* y diseño de heurísticas, todos los movimientos válidos en estos mapas tienen costo uniforme de 1.

Para las heurísticas base de este taller, `manhattanHeuristic` y `euclideanHeuristic` deben calcular la distancia directa desde la posición actual hasta el objetivo de referencia de la fase: K si el robot todavía no tiene el kit, el sistema T pendiente más cercano si el robot ya tiene el kit y aún quedan sistemas por reparar, y C si ya no quedan sistemas pendientes. La heurística propia deberá considerar de forma más completa el costo restante de la misión, utilizando la relación entre el kit, los sistemas pendientes y el centro de control.

- a) Implemente `aStarSearch` en `algorithms/search.py`.
- b) Implemente `manhattanHeuristic` en `algorithms/heuristics.py`.
- c) Implemente `euclideanHeuristic` en `algorithms/heuristics.py`.
- d) Diseñe e implemente `systemRepairHeuristic` en `algorithms/heuristics.py`. La heurística diseñada debe ser admisible y consistente, debe retornar 0 en los estados meta y debe usar información relevante del estado, como la ubicación del kit, los sistemas pendientes y el centro de control.
- e) Para esta misión, utilice la clase `SystemRepairProblem` y pruebe `aStarSearch` con las diferentes heurísticas sobre los mapas disponibles en `layouts/systems`, donde aumenta progresivamente la cantidad de sistemas T.
- f) Compare `nullHeuristic`, `manhattanHeuristic`, `euclideanHeuristic` y `systemRepairHeuristic` en términos de costo de la solución, cantidad de nodos expandidos y tiempo de ejecución. Conserve estos resultados para el análisis del punto 6.

Punto 6: Reflexión sobre el trabajo realizado (10 %)

6a. Análisis de los algoritmos y heurísticas (6 %)

Para DFS, BFS, UCS y A*, responda de manera argumentada los siguientes temas. Use resultados obtenidos en mapas relevantes del taller cuando sea útil para sustentar sus conclusiones.

- Complejidad temporal y espacial de cada algoritmo (Notación O).
- Completitud de cada algoritmo en el contexto de los problemas del taller.
- Condiciones bajo las cuales cada algoritmo encuentra una solución óptima.
- Explique por qué la posición del robot no es suficiente para representar el problema del punto 4 y por qué `hasModule` debe formar parte del estado.
- Consistencia de las heurísticas `manhattanHeuristic`, `euclideanHeuristic` y su heurística propia.
- Compare `nullHeuristic`, `manhattanHeuristic`, `euclideanHeuristic` y su heurística propia en términos de costo de solución, nodos expandidos y tiempo de ejecución.

Se espera que en su análisis compare cualitativamente el desempeño de los distintos algoritmos en los layouts que encuentre de interés, así como que compare cuantitativamente métricas como el costo de la solución, la cantidad de nodos expandidos y el tiempo.

Algunas instancias del problema de reparación de múltiples sistemas pueden tener un espacio de búsqueda particularmente grande. Se recomienda inspeccionar los mapas disponibles en `layouts/systems` y comenzar las pruebas con aquellos que contengan una menor cantidad de sistemas averiados. Si su heurística no permite resolver alguna instancia en un tiempo razonable, explique las posibles causas.

6b. Reflexión sobre el uso de inteligencia artificial (4 %)

Escriba una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA (vea política de uso de la IA más adelante), indicando, por ejemplo, qué aprendizajes obtuvo al ver las correcciones hechas por la IA, si éstas fueron básicas o solo mejoras secundarias; la facilidad o dificultad para crear el prompt adecuado, si la solución propuesta fue fácil de entender (¿usted la podría explicar en una sustentación?). Si no hizo ninguna iteración con IA, indique por qué no le pareció necesario ni útil. Aquí no hay respuestas correctas ni incorrectas. Este punto busca que usted se tome el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

Política de uso de inteligencia artificial

El uso de herramientas de inteligencia artificial deberá realizarse de forma responsable y documentada. Primero, los estudiantes deberán desarrollar una versión autónoma de la solución. Posteriormente, podrán utilizar herramientas de IA para apoyar la revisión, corrección, depuración, explicación o mejora del código.

Para cada función o fragmento de código trabajado con apoyo de IA, deberán incluir en los anexos del PDF la versión inicial correspondiente, los prompts utilizados, las respuestas o sugerencias relevantes, los cambios aceptados, rechazados o adaptados, y una breve explicación de cómo verificaron su funcionamiento.

Los archivos del proyecto deberán contener únicamente la versión final activa del código. Los anexos deberán organizarse por punto, archivo o función, de manera que sea posible identificar con claridad el proceso seguido en cada caso. Si no se utilizaron herramientas de inteligencia artificial, deberá indicarse explícitamente en el documento de reflexión.

Entrega del trabajo

- Guarde todo su trabajo (incluyendo el documento PDF) en una carpeta comprimida (.zip)
- Suba el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerde incluir los nombres y códigos de todos los integrantes de su grupo.

Nota: Al enviar su solución, usted declara que las versiones iniciales incluidas en los anexos son de su autoría, que las versiones desarrolladas con apoyo de IA corresponden a los prompts registrados, que comprende el funcionamiento del código final y que acepta que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.